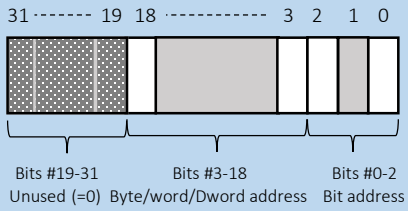
Indirect Addressing

- Indirect addressing is a common technique in programming
- Use of pointers to refer to operands
- Allows to implement iterative cycles
- Available for the instruction list programming language of (Siemens) PLCs
- We want to implement operations like:

```
Pointer_1 = N; // Initializes Pointer_1 to N
L EW[Pointer_1];  // brings the content of EWN in ACCU1
******
Pointer_2 = K; // Initializes Pointer_2 to K
L 50; //Load integer 50 in ACCU1
T MW[Pointer_2]; // transfers 50 in MWK
```

```
Pointer_1 = i.j; // Initializes Pointer_1 to i.j
U E[Pointer_1] // Executes the U operation on input bit  Ei.j
******
Pointer_2 = h.k; // Initializes Pointer_2 to h.k
O M0.0; // OR operation on bit M0.0
S A[Pointer_2]; // if M0.0=1 sets output bit  Ah.k
```

Pointer format



32 bit long

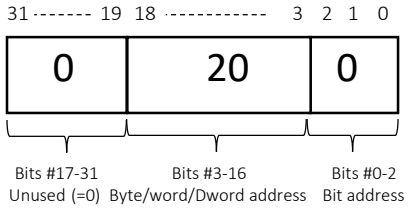
Bits #0-2 used to (indirectly) address the bit

Bits #3-18 used to (indirectly) address byte/word/Dword

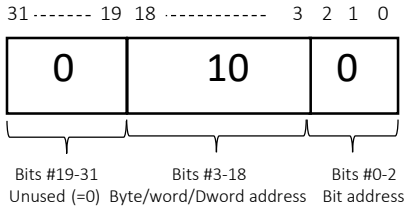
Bits #19-31 unused, set to zero

Examples of Indirect Addressing

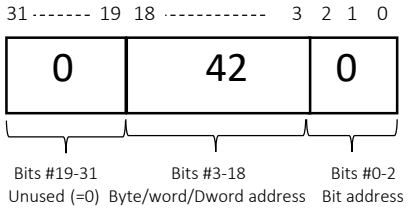
- Let's suppose we have been able to bring in ACCU1 the following bit patterns that represent different pointer values



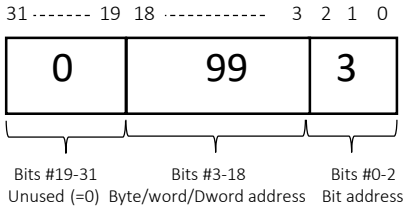
T MD0; // Copy ACCU1 content to MD0
L EW[MD0] // Brings the content of **EW20** in ACCU1



T MD16; // Copy ACCU1 content to MD16
L 19;
T AW[MD16] // Writes the value 19 in **AW10**



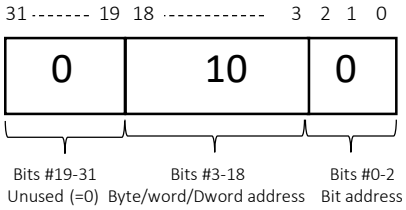
T MD20; // Copy ACCU1 content to MD20
L EB[MD20] // Brings the content of **EB42** in ACCU1



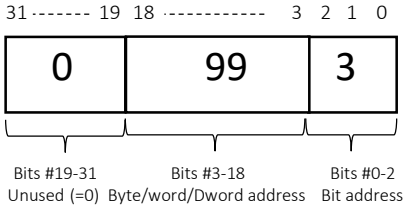
T MD24; // Copy ACCU1 content to MD24
U M2.2;
U M[MD24]; // AND operation between M2.2 and **M99.3**

Pointer Format

- The statement list language of Siemens PLCs **has a specific format for pointers** that can be easily used
- The format is **P#x.y** where **x identifies the byte/word/Dword** and **y identifies the bit**
- Example: the instruction L P#10.0 brings in ACCU1 the following bit pattern



- The instruction L P#99.3 brings in ACCU1 the following bit pattern



Examples of Pointer Format Use

L P#20.0 // Brings pointer value P#20.0 in ACCU1
T MD0; // Copy ACCU1 content to MD0
L EW[MD0] // Brings the content of **EW20** in ACCU1
(equivalent to the instruction L EW20)

L P#10.0 // Brings pointer value P#10.0 in ACCU1
T MD16; // Copy ACCU1 content to MD16
L 19;
T AW[MD16] // Writes the value 19 in **AW10**
(equivalent to the instruction T AW10)

L P#42.0 // Brings pointer value P#42.0 in ACCU1
T MD20; // Copy ACCU1 content to MD20
L EB[MD20] // Brings the content of **EB42** in ACCU1
(equivalent to the instruction L EB42)

L P#99.3 // Brings pointer value P#99.3 in ACCU1
T MD24; // Copy ACCU1 content to MD24
U M2.2;
U M[MD24]; // AND operation between M2.2 and **M99.3**

POINTERS CAN BE INCREMENTED

L P#0.0 // Brings pointer value P#0.0 in ACCU1
T MD0; // Copy ACCU1 content to MD0
L EW[MD0] // Brings the content of **EW0** in ACCU1
L P#2.0 // Pointer increment (word)
L MD0
+D // N.B. Pointer is a 32-bit number, **+D** must be used
T MD0
L EW[MD0] // Brings the content of **EW2** in ACCU1

L P#99.3 // Brings pointer value P#99.3 in ACCU1
T MD24; // Copy ACCU1 content to MD24
U M2.2;
U M[MD24]; // AND operation between M2.2 and **M99.3**
= M0.0
L P#0.1 Pointer increment (bit)
L MD24
+D // N.B. Pointer is a 32-bit number, **+D** must be used
T MD24
U M2.2;
U M[MD24]; // AND operation between M2.2 and **M99.4**

16-bit Pointer Format

- A 16-bit pointer is also available for a **different set of** indirect addressing operations
- A 16 bit number **stored on a word**, can be used as pointer for example to, indirectly,
  - Open a data block
  - Conditionally call a function
  - Start a timer

```
L 100 // Brings (pointer) value 100 in ACCU1
T MW80; // Copy ACCU1 content to MW80
AUF DB[MW80] // Opens DB100
```

```
L 1 // Brings (pointer) value 1 in ACCU1
T MW0; // Copy ACCU1 content to MW0
SE T[MW0] // Starts Timer T1 with modality SE.
N.B. With modality SE, timer starts only if RLO = 1
```

```
L 11// Brings (pointer) value 11 in ACCU1
T MW10; // Copy ACCU1 content to MW10
CC FC[MW10] // Call FC11
CC actually calls the function if RLO = 1
N.B. Indirect call of a FC is only possible for
FCs without parameters
```

Suggested Exercises

- Calculation of FC7 execution times (use the S7-300 Instruction List Manual, available on the course site, where the instructions execution times are provided)
- Implementation of FC7 variants in which
  - Reset operation is carried out on Data Words (DBW)
  - Reset operation is carried out on Double Data Words (DBD)
- Calculation of the FC7 variants execution times
- Implementation of a routine that calls FC6 where the parameter ACQ is the input word EW124 and the status byte is AB124